
HASH TABLES

Assoc.Prof. Suphakant Phimoltares, Ph.D.
Advanced Virtual and Intelligent Computing (AVIC) Center

Hash Tables

- A **hash table** is a data structure that supports the following operations:
 - Hash-Insert(T, k) - puts key k into the hash table T
 - Hash-Search(T, k) - searches for key k in the hash table T
 - Hash-Remove(T, k) - removes key k from the hash table T
- In a well-formed hash table, each of these operations take on average $O(1)$ time, making hash tables a very useful data structure.
- **Hashing** is a technique that retrieves the value using the index obtained from key without performing a search.

Hash Tables

Hash Table

0	
1	
2	93
3	
4	
5	40
6	76
7	
8	
9	

Hash Functions

- The function that maps a **key** to an **index** in the hash table is called a **hash function**.
- The goal of a hash function, h , is to map each key k to an integer in the range $[0, N - 1]$, where N is the capacity of the bucket array for a hash table.
- Example: A hash function for integer keys

$$h(x) = x \bmod N$$

Collision Handling

- A collision occurs when two keys are mapped to the same index in a hash table. Generally, there are two ways for handling collisions:
 - **Open addressing**: the colliding item is placed in a different cell of the table.
 - linear probing
 - quadratic probing
 - double hashing
 - **Separate chaining**: places elements that hash to the same cell in a **linked list**.

Open Addressing

HASH-INSERT(T, k)

```
1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == \text{NIL}$ 
5           $T[j] = k$ 
6          return  $j$ 
7      else  $i = i + 1$ 
8  until  $i == m$ 
9  error “hash table overflow”
```

HASH-SEARCH(T, k)

```
1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == k$ 
5          return  $j$ 
6       $i = i + 1$ 
7  until  $T[j] == \text{NIL}$  or  $i == m$ 
8  return NIL
```

Deletion uses the special value DELETED instead of NIL.

Linear Probing

- Linear probing handles collisions by placing an item in the first available cell of the following series.

$$h(k, i) = (h'(k) + i) \bmod N \text{ for } i = 0, 1, \dots, N - 1$$

- Each table cell inspected is referred to as a “**probe**”
- **Example:** consider a hash table storing integer keys that handles collision with linear probing
 - $N = 7$
 - $h'(k) = k \bmod 7$
 - Insert keys 76, 93, 40, 47, 10, 55, in this order

Linear Probing Example

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

Quadratic Probing

- Quadratic probing handles collisions by placing an item in the first available cell of the following series.

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod N \text{ for } i = 0, 1, \dots, N - 1$$

- **Example:** consider a hash table storing integer keys that handles collision with quadratic probing
 - $N = 7$
 - $h'(k) = k \bmod 7, c_1=0, c_2 = 1$
 - Insert keys 76, 93, 40, 47, 10, 55, in this order

Quadratic Probing Example

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

Double Hashing

- Double hashing uses a secondary hash function $d(k)$ and handles collisions by placing an item in the first available cell of the following series.

$$h(k, i) = (h(k) + id(k)) \bmod N \text{ for } i = 0, 1, \dots, N - 1$$

- The common choice for the secondary hash function is

$$d(k) = q - k \bmod q$$

where $q < N$ and q is a prime.

Double Hashing

- The secondary hash function $d(k)$ cannot have zero values.
- The table size N must be a prime to allow probing of all the cells
- Example: consider a hash table storing integer keys that handles collision with double hashing
 - $N = 7$
 - $h(k) = k \bmod 7$
 - $d(k) = 5 - k \bmod 5$
 - Insert keys 76, 93, 40, 47, 10, 55, in this order

Double Hashing Example

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

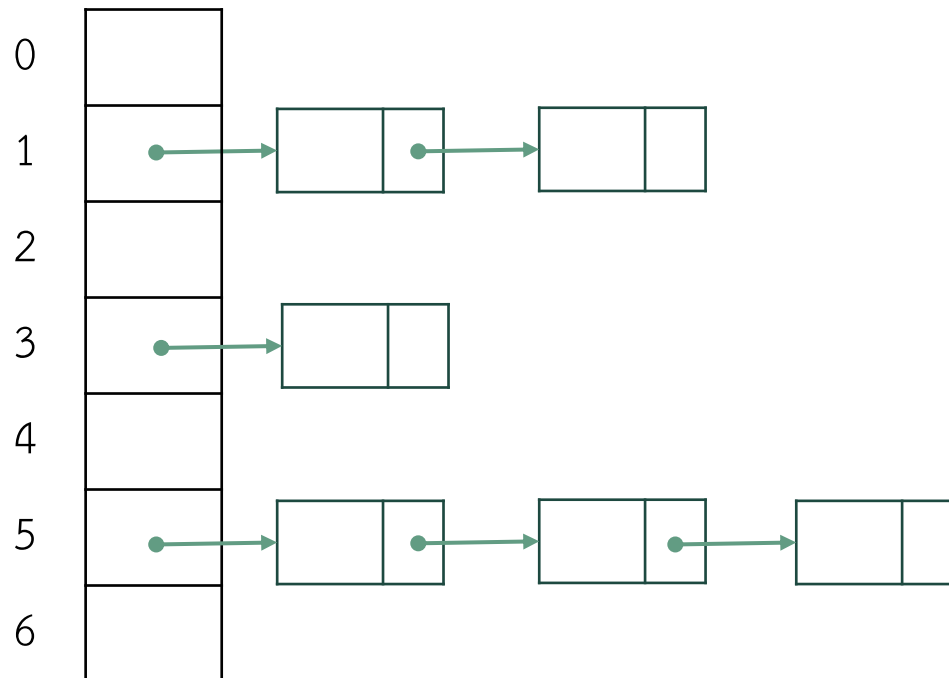
0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

0	
1	
2	
3	
4	
5	
6	

Separate Chaining

- The separate chaining scheme places all entries with the same hash index into the same location, rather than finding new locations. Each location in the separate chaining scheme is called a **bucket**. A bucket is a container that holds multiple entries.



Separate Chaining

- Example: consider a hash table storing integer keys that handles collision with chaining
 - $N = 7$
 - $h(k) = k \bmod 7$
 - Insert keys 76, 93, 40, 47, 10, 55, 62 in this order

0	
1	
2	
3	
4	
5	
6	

References

1. T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd edition, The MIT Press, 2009.
2. M.T. Goodrich and R. Tamassia, Algorithm Design and Applications, Wiley, 2015.